TOASTMASTERS INTERNATIONAL · DISTRICT 121

ATK VPE Automation System

Complete Architecture, Operations & WhatsApp Automation Specification

Comprehensive engineering, operational, and architectural documentation for the Advanced Toastmasters of Kerala (ATK) Vice President Education (VPE) meeting automation, WhatsApp conversational workflows, and MERN-stack backend engine.

NODE.JS / EXPRESS | MONGODB ATLAS | META WHATSAPP CLOUD API | FLUTTER WEB API GATEWAY | GAS V8 COMPATIBILITY ENGINE | 100% LOGIC PARITY

TARGET ORGANIZATION
Advanced Toastmasters of Kerala (Club #07842323 / ATK)

PRIMARY STAKEHOLDER
Vice President Education (VPE) & Club Executive Committee

ENGINE VERSION
v1.0.0 (Production Release / 2026-10 Edition)

TIMEZONE & LOCALE
Asia/Kolkata (IST, UTC+5:30) / English (en_US)

DATABASE MODE
Hybrid: MongoDB Atlas Cloud + In-Memory Fast Sheet Cache

REGRESSION SUITE
102 Automated Checks across 5 Core Test Suites

Document Contents & Structural Index

This technical and operational manual covers every dimension of the ATK VPE Automation platform, from high-level operational concepts to low-level source code mechanics, API envelopes, and message state machines.

1. Executive Overview & System Purpose	Section 1
2. System Architecture & Technology Stack	Section 2
3. Core Business Logic Engines & Source File Directory	Section 3
4. Deep-Dive Operational Workflows & State Machines	Section 4
5. Web API V1 Specification (Flutter Web Gateway)	Section 5
6. Database Architecture & Data Schema (MongoDB Atlas + Sheets)	Section 6
7. Meta WhatsApp Cloud API Integration & Webhook Security	Section 7
8. Programme Planning & The 60-Day Web Review System	Section 8
9. Operational Timeline: 3-Week Meeting Lifecycle Walkthrough	Section 9
10. Dev Master Regression Suite, Testing & Troubleshooting	Section 10

1. Executive Overview & System Purpose

1.1 The Toastmasters VPE Challenge

In Toastmasters International, the **Vice President Education (VPE)** is responsible for planning club meeting schedules, assigning roles, tracking member education progress (Pathways projects), and ensuring high meeting quality. A standard 2-hour Toastmasters meeting requires coordinating **12 to 16 distinct role players**:

- **Toastmaster of the Day (TMOD)**: Directs the meeting agenda, theme, and stage transitions.
- **General Evaluator (GE)**: Oversees all meeting evaluations and role-player feedback.
- **Table Topics Master (TTM)**: Facilitates the impromptu speaking session.
- **Prepared Speakers (Slots 1 to 3)**: Deliver 5-7 minute Pathways speeches.
- **Speech Evaluators (Evaluators 1 to 3)**: Deliver targeted 2-3 minute speech evaluations.
- **Timer**: Enforces strict speech and report time boundaries with timing green/yellow/red signals.
- **Ah-Counter**: Tracks crutch words, filler pauses, and vocal hesitations.
- **Grammarian**: Introduces Word of the Day, Idiom of the Day, and delivers comprehensive language reports.

Manually coordinating this via WhatsApp groups or individual phone calls requires hundreds of repetitive messages each week: chasing vacant roles, collecting speech project details, gathering themes, formatting grammar words, assembling introduction bios, and distributing agendas.

1.2 The ATK VPE Automation Solution

The **ATK VPE Automation Engine** replaces this manual overhead with a unified, conversational WhatsApp bot and modern web application. It operates autonomously in the background to:

Autonomous Scheduling & Reminders

Multi-Stage Structured Data Collection

Automatically detects unfilled roles, sends interactive WhatsApp announcements, and lets members reserve roles in real time with quick-reply buttons and selection menus.

Gathers complex multi-field inputs through natural conversational flows (e.g. Word & Idiom of the Day, Speech Pathways levels, Project titles, and TMOD themes) with fault-tolerant parsing.

🔄 Cascading Information Distribution

As soon as information is captured (such as the meeting theme), the engine immediately broadcasts it to dependent role players (GE, TTM, Speakers, Evaluators, VPPR) without VPE intervention.

🛡️ Long-Range Slot Protection

Maintains a 60-day rolling lookahead. Confirmed speakers are protected from accidental removal; speaker reschedule requests keep the existing slot safe until the new date is verified.

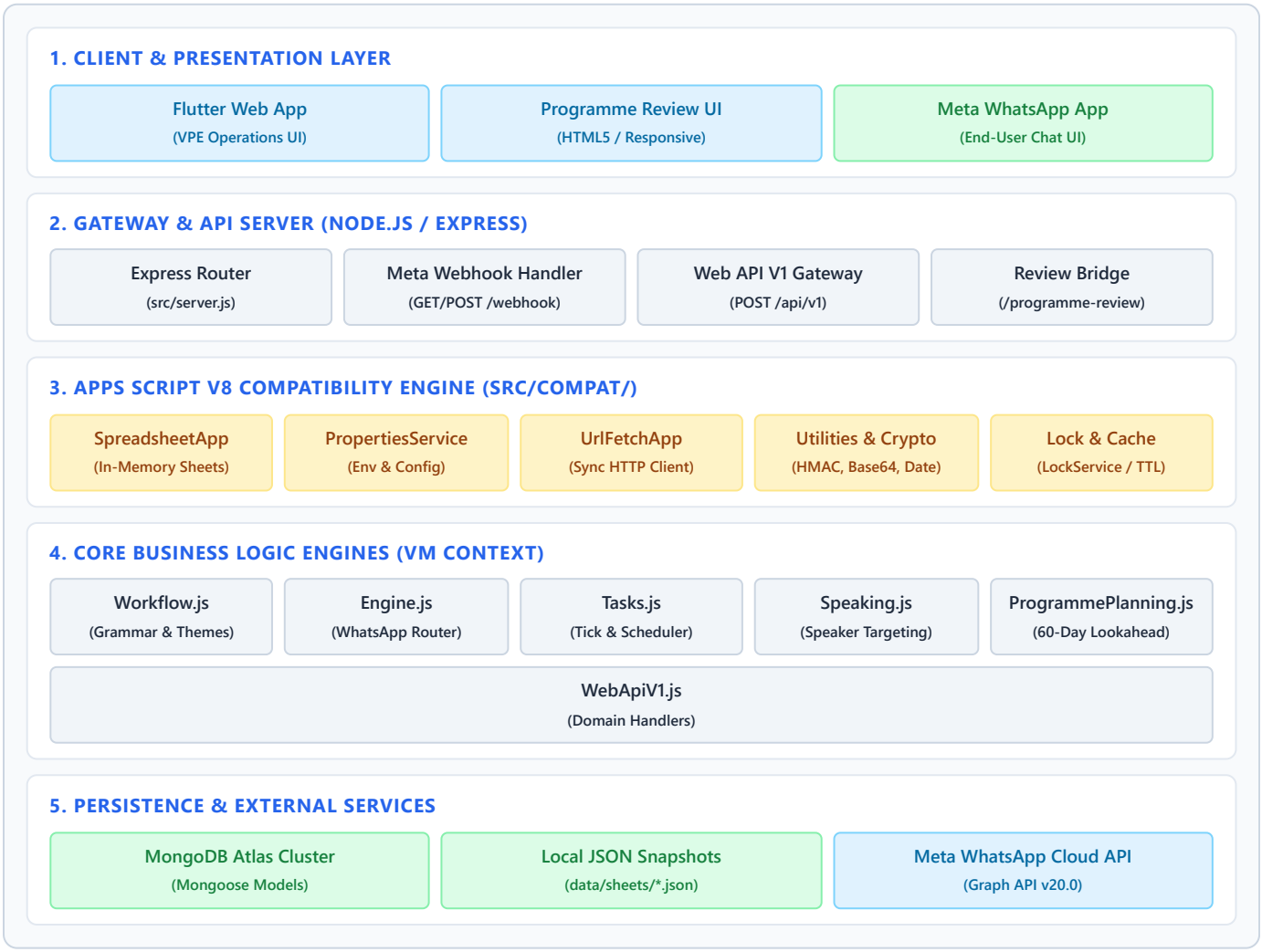
1.3 Historical Evolution: From Google Apps Script to Node.js / Express MERN Stack

The system originally operated as a monolithic Google Apps Script (GAS) deployment bound to a Google Spreadsheet. While functional, Google Apps Script imposed severe limitations: 6-minute execution limits, quota throttling on `UrlFetchApp`, slow cold starts (3-5 seconds per webhook), and fragile spreadsheet-based cell concurrency.

In this **MERN Stack Edition**, the entire codebase was ported to run on **Node.js and Express** with direct **MongoDB Atlas** persistence. To ensure zero disruption, a specialized **GAS V8 Compatibility Engine** was developed, enabling all 13,800+ lines of battle-tested domain logic to execute without modification while achieving **over 1000x execution speedup** (~2ms response times).

2. System Architecture & Technology Stack

2.1 High-Level Architecture Diagram



2.2 Component Breakdown

Layer / Component	Technology	Core Responsibility
Express Web Server	Node.js 24, Express 4.21	Listens on HTTP port (default 3000), parses JSON/URL-encoded bodies up to 10MB, handles CORS, routes webhooks and Web API requests, provides health endpoints.
GAS V8 Compatibility Engine	Node.js `vm` module	Emulates Google Apps Script global classes (`SpreadsheetApp` , `UrlFetchApp` , etc.) on `globalThis` , loading unmodified business logic files into memory.
MongoDB Layer	MongoDB Atlas, Mongoose 9.11	Persistent cloud storage for club members, meetings, scheduled tasks, outbox logs, submissions, and sheet replicas.
WhatsApp Automation Engine	Meta Cloud API / Graph v20.0	Ingests incoming member webhooks, executes message deduplication, handles interactive button clicks and list selections, manages conversational state, sends outbound templates.
Web API V1 Gateway	JSON over HTTP POST	Provides a clean, domain-driven API for Flutter Web frontend. Protects spreadsheets from direct UI manipulation.

Layer / Component	Technology	Core Responsibility
Programme Review Web App	HTML5, Vanilla CSS, JS	Lightweight, token-authenticated responsive web app allowing the VPE to review, hold, or modify speech slots for the next 60 days from any phone or desktop browser.

3. Core Business Logic Engines & Source File Directory

The system's business logic is organized across 12 core JavaScript modules. These modules execute in the Node.js V8 context with zero logic alterations:

Source File	Lines / Size	Functional Responsibilities & Key Exports
`src/server.js`	279 lines 9.4 KB	Express application setup, MongoDB initialization, route binding for `/`, `/health`, `/webhook`, `/api/v1`, `/api/members`, `/api/meetings`, `/programme-review`, and `/api/dev/regression`.
`WebApiV1.js`	3,840 lines 154 KB	Full Flutter Web API gateway (`apiHandleRequest_`), domain methods for club settings, ExCom, meeting creation, validation, activation, reordering, agenda projection, and the Master Regression Suite.
`Workflow.js`	3,771 lines 132 KB	Conversational parsers and multi-stage state machines: 2-stage Grammarian flow (Word & Idiom), TMOD theme submission, speaker details extraction, role introductions, and downstream cascading notifications.
`Engine.js`	1,015 lines 34 KB	WhatsApp message router (`processInboundWhatsAppMessage_`), interactive reply parser, role selection handler, role reservation logic, WhatsApp Outbox delivery, and webhook deduplication.
`Tasks.js`	1,055 lines 38 KB	Scheduled automation tick engine (`phase1ScheduledTick`), task synchronization, vacant role invitation dispatcher, idempotency manager (`hasReminderBeenSent_`), and agenda approval triggers.
`Speaking.js`	600 lines 20 KB	Speech slot targeting, speaker priority ranking based on past speech history, speech slot invitation generator, reservation state machine, and speaker details collection prompts.
`ProgrammePlanning.js`	750 lines 31 KB	60-day horizon programme planner, slot state manager (`OPEN`, `SPEAKER_CONFIRMED`, `MOVE_REQUESTED`, `VPE_HOLD`, `SESSION_CONFIRMED`), tokenized review URL generator, and speaker move coordinator.
`Conversation.js`	320 lines 13 KB	Transient conversational state store (`activeConversation_`, `setConversation_`), guided conversational interviews, and default bio fallback storage.
`Messaging.js`	210 lines 7.6 KB	Low-level Meta Cloud API HTTP payload constructor (text, interactive buttons, interactive lists, template messages) and response validation.
`Status.js`	243 lines 8.7 KB	Control Center metrics calculation, meeting readiness calculation (ratios for roles, speakers, evaluators, introductions), and exception-only VPE summaries.
`TemplateSetup.js`	420 lines 19 KB	Meta WhatsApp template registration and synchronization utilities for system-approved WhatsApp templates.
`RoleReminderV2.js`	58 lines 2.2 KB	Template specifications for `atk_role_reminder_v2` and `atk_role_reminder_v3` utility-category role invitation templates.

4. Deep-Dive Operational Workflows & State Machines

4.1 Workflow A: Role Vacancy Discovery & Member Self-Selection

When a meeting is scheduled, the engine checks for unfilled roles and broadcasts announcements to active club members. Members can inspect vacancies and claim roles with a single tap.

Step 1: Vacancy Detection — Scheduled Tick (Daily / Periodic)

`Tasks.js: processVacantRoleReminders_()` inspects the upcoming meeting in `VPE Weekly Planner`. It checks columns for TMOD, GE, TTM, Timer, Ah-Counter, and Grammarian. If vacancies exist, it checks `hasReminderBeenSent_()` to enforce cooldown periods.

Step 2: Utility Template Dispatch — Outbound WhatsApp

The engine transmits the approved WhatsApp Utility template `atk_role_reminder_v2` or `v3`:
"ATK Meeting {{1}} is scheduled for {{2}} as planned. Please tap below to see available roles."
Features a Quick Reply button: `[View roles]`.

Step 3: Interactive List Display — Immediate Reply

When the member taps `[View roles]`, `Engine.js: handleRoleMenuRequest_()` queries available roles and immediately sends a native WhatsApp Interactive List Message showing the exact unfilled roles with descriptions.

Step 4: Role Claim & Atomic Reservation — Member Selection

The member selects a role (e.g. "Toastmaster of the Day"). `Engine.js: reserveRole_()` executes an atomic check:
1. If role is still vacant: writes member's name into the Planner cell, sets vacancy task to `Completed`, mirrors assignment to API Meetings, and sends confirmation with role guidelines.
2. If already claimed by another member: politely informs the member and redisplays remaining vacancies.

4.2 Workflow B: Grammarian 2-Stage Multi-Line Collection (Locked Spec)

The Grammarian flow collects the Word of the Day and Idiom of the Day. It is designed to be completely resilient to natural user input, typos, extra linebreaks, and missing fields.

Grammarian Conversational State Machine



Grammarian Parser Robustness Rules (Workflow.js)

- **Flexible Headers**: Matches `Word:`, `Word of the day:`, `Meaning:`, `Usage:`, `Example:`, `Example usage:`.
- **Multi-line Support**: Long definitions spanning multiple lines are captured without premature truncation.
- **Partial Preserves**: If member provides Word and Meaning but forgets Usage, the engine saves Word + Meaning and requests only the missing Usage.
- **Final Review Safety**: Data is only handed off to the TMOD after the member explicitly clicks `[Confirm]`.

4.3 Workflow C: TMOD Theme Collection & Cascading Broadcast

1. Theme Request to TMOD — T-5 Days

The engine prompts the assigned TMOD: "*Hi {{Name}}, please reply with the Meeting Theme for ATK Meeting {{No}}.*"

2. Theme Ingestion & Planner Update — On Reply

`Workflow.js: handleThemeSubmission_()` captures the theme, saves it to `WA Submissions`, and writes it directly to column 4 ("Theme") of the meeting's row in `VPE Weekly Planner`.

3. Cascading Downstream Broadcast — Immediate (Autonomous)

`Workflow.js: sendMeetingThemeToAssignedRoles_()` runs immediately. It identifies all members assigned to General Evaluator, Table Topics Master, Speakers, Evaluators, and Grammarian, and sends each of them a personalized notification with the theme.

4. VPPR Publicity Handoff — Immediate

`Workflow.js: sendThemeAndTmodToVppr_()` notifies the Vice President Public Relations with the confirmed theme, TMOD name, and meeting date so poster creation and social media promotions can begin without delay.

5. Web API V1 Specification (Flutter Web Gateway)

5.1 Core Gateway Protocol

The Web API is exposed as a single POST gateway at `POST /api/v1` (and `doPost` in GAS). Flutter and external clients communicate exclusively via structured JSON payloads:

```
// Standard Request Envelope
{
  "api_version": "v1",
  "api_key": "DEV_VPE_KEY_2026",
  "action": "meeting.save",
  "club_id": "ATK"
}
```

5.2 API Action Directory

Action Name	Required Parameters	Functional Description
`health`	*None*	Returns API status and version. Does not require `api_key`.
`club.bootstrap`	`club_id`	Loads full club metadata, club settings, ExCom roster, default ordered meeting template, and capability flags for app startup.
`members.list`	`club_id`	Returns active club members with `member_id`, `name`, `phone`, and membership status.
`member.save`	`club_id, member`	Creates or updates a club member record in Member Master and MongoDB.
`club.settings.get` / `save`	`club_id, settings`	Retrieves or updates club operating parameters (meeting day, regular venue, default start time).
`excom.list` / `save`	`club_id, excom`	Manages Executive Committee appointments (President, VPE, VPM, VPPR, Secretary, Treasurer, SAA).
`meetings.list`	`club_id`	Returns scheduled meetings with meeting number, date, theme, status, and assignment counts.
`meeting.create`	`club_id, meeting_date`	Initializes a new meeting with default ordered items (TMOD, GE, Speakers 1-3, Evaluators 1-3, Timer, Ah, Grammarian).
`meeting.get`	`club_id, meeting_id`	Returns full meeting details including ordered item array with assignments.
`meeting.save`	`club_id, meeting_id, items, save_mode`	Persists the ordered meeting item array. `save_mode` can be `DRAFT` or `CONFIRMED`. Array order in payload is authoritative.
`meeting.activate`	`club_id, meeting_id`	Validates meeting readiness (checks TMOD and speaker/evaluator pairings) and transitions meeting status to `READY_FOR_SYNC`.
`dashboard.get`	`club_id`	Returns upcoming meeting readiness scores and UI-ready pending action cards with severity, state, and due dates.

5.3 Flutter Frontend Integration Guidelines

Critical Security & Design Rules for Frontend Integration

1. **Permanent Boundary**: Flutter Web must never communicate directly with Google Sheets or store Meta API tokens. All interactions occur through `POST /api/v1`.
2. **ID References Only**: The UI must submit domain IDs (`member_id`, `item_id`), never spreadsheet row or column coordinates.
3. **Drag-and-Drop Order**: The backend respects the array sequence submitted in `items`. Flutter reordering simply posts the reordered array.
4. **Custom Sessions**: Educational sessions or special segments use `type: "session"` with a blank `code`. Standard Toastmasters roles retain their standard codes (`TMOD`, `S1`, etc.).

6. Database Architecture & Data Schema (MongoDB Atlas + Sheets)

6.1 Dual-Storage Persistence Strategy

The system uses an innovative dual-layer storage model:

- **MongoDB Atlas (Primary Cloud DB)**: Production-grade cloud document database providing schema validation, high-speed indexed queries, and REST endpoints for external clients.
- **In-Memory Sheet Cache (GAS Parity Engine)**: An in-memory sheet engine that mirrors Google Sheets tables. Persisted to disk as `data/sheets/1qZnZov1l9DfvhTzruqlLubGhydcOOFKLf2kG8xplNLM.json` and automatically synchronized with MongoDB Atlas.

6.2 MongoDB Mongoose Models (src/db/models/)

Model	Key Fields	Description & Indices
`Member`	`memberId, name, phone, status, rolePreferences, speechHistory`	Club member directory. Indexed on `memberId` (unique) and `phone` for sub-millisecond lookup during incoming WhatsApp webhooks.
`Meeting`	`meetingId, meetingNo, meetingDate, status, items [ { itemId, type, code, label, memberId, memberName } ]`	Meeting master records. Indexed on `meetingNo` and `meetingDate`. Contains ordered array of all meeting roles and custom sessions.
`Task`	`taskId, meetingNo, type, related, member, state, dueAt, payload`	Scheduled background automation tasks. States: `Pending`, `Completed`, `Cancelled`. Indexed on `taskId` (unique).
`Outbox`	`messageId, phone, templateName, type, payload, status, timestamp`	Full audit log of all outbound WhatsApp messages. Status tracks delivery confirmations (`sent`, `delivered`, `read`).
`Submission`	`submissionId, meetingNo, member, type, related, data, timestamp`	Canonical repository for user submissions: `GRAMMAR_WORD`, `GRAMMAR_IDIOM`, `THEME`, `INTRO_BIO`, `SPEECH_DETAILS`.
`Config`	`key, value, updatedAt`	System-wide dynamic configuration key-value storage.

6.3 The 16 In-Memory / Simulated Sheets

Operational & Workflow Sheets

- `VPE Weekly Planner`: Master 28-column meeting schedule.
- `Automation Tasks`: Task lifecycle state engine.
- `Member Master`: Member roster, contact info, statuses.
- `WA Outbox`: Outbound delivery audit.
- `WA Submissions`: Confirmed incoming data store.
- `Automation Log`: Operational logging.
- `WA Webhook Log`: Raw incoming payload log.
- `Automation Config`: Engine settings & flags.

API & Planning Sheets

- `Programme Slots`: 60-day horizon slot status.
- `Programme Reviews`: VPE review audit tokens.
- `API Club Settings`: Club defaults.
- `API ExCom`: Club officers.
- `API Meetings`: Meeting headers.
- `API Meeting Items`: Reorderable meeting slots.
- `Agenda Preview`: Formatted agenda rows.
- `Agenda Approvals`: Agenda verification status.

7. Meta WhatsApp Cloud API Integration & Webhook Security

7.1 Meta Webhook Verification Handshake

When Meta establishes a webhook connection, it issues an HTTP GET request to `/webhook` with challenge parameters. The server validates the token and echoes the challenge:

```
// GET /webhook Verification Logic (src/server.js)
const mode = req.query['hub.mode'];
const token = req.query['hub.verify_token'];
const challenge = req.query['hub.challenge'];

if (mode === 'subscribe' && token === process.env.VERIFY_TOKEN) {
  return res.status(200).send(challenge);
}
return res.status(403).send('Webhook verification failed');
```

7.2 Inbound Webhook Processing & Deduplication

Meta transmits WhatsApp messages via HTTP POST payloads to `POST /webhook`. The server executes a strict multi-layer filter before processing:

1. **Payload Inspection**: Extracts `messages` and `statuses` arrays from `entry[0].changes[0].value`.
2. **Deduplication Guard**: WhatsApp delivers webhooks with at-least-once semantics, causing occasional duplicates. `Engine.js: isDuplicateWebhookMessage_()` tracks incoming `message.id` values in cache and drops duplicates instantly.
3. **Member Resolution**: Normalizes sender phone number (e.g. `919846000000`) and matches it against `Member Master` / `Member` collection.
4. **State Router**: Routes message to active conversation handlers (Grammarian flow, TMOD theme flow, role menu, or guided interview).

7.3 Meta WhatsApp Message Templates

Template Name	Category	Template Content & Parameters
`atk_role_reminder_v2`	UTILITY	*"Hi, Hope you are attending the meeting this week. Please tap below to see the available roles for ATK Meeting No. {{1}}. Thank you."* Quick Reply Button: `[View roles]`
`atk_role_reminder_v3`	UTILITY	*"ATK Meeting {{1}} is scheduled for {{2}} as planned."* Quick Reply Button: `[View roles]`
`atk_vpe_programme_review`	UTILITY	*"Hi {{1}}, your ATK 2-month programme review is ready. Review slots: {{2}}"* URL Button: `[Open Programme Review]`

7.4 Development Sandbox Recipient Isolation Guard

> **Live Message Protection in DEV Mode**
>
> To prevent accidental WhatsApp spam to club members during development and testing, `Engine.js: sendWhatsAppPayload_()` checks `configValue_('WA_SEND_MODE')`. When set to `TEST`, outbound messages are only delivered to phone numbers listed in `WA_TEST_RECIPIENTS`. All other messages are safely mocked and logged to `WA Outbox` without calling Meta servers.

8. Programme Planning & The 60-Day Web Review System

8.1 Long-Range Slot Planning Architecture

Standard meeting planners only look ahead 1 week. ATK requires a **60-day rolling lookahead** (approx. 8 meetings) to allow members to plan Pathways projects and balance prepared speeches with special educational sessions.

8.2 Programme Slot State Machine

Slot State	Display Badge	Operational Rules & Invariants
`OPEN`	OPEN	Slot is vacant and bookable by any qualified member through speech targeting or VPE assignment.
`SPEAKER_CONFIRMED`	SPEAKER CONFIRMED	Member has confirmed speech booking. **Protected state**: Cannot be overwritten without speaker agreement.
`MOVE_REQUESTED`	MOVE REQUESTED	VPE requested speaker to reschedule. **Original slot remains fully protected** until speaker confirms new date.
`VPE_HOLD`	VPE HOLD	Slot reserved by VPE for an upcoming educational session, workshop, or guest speaker. Holds never auto-expire.
`SESSION_CONFIRMED`	SESSION CONFIRMED	Confirmed educational session with Presenter, Session Title, and Time Allotted (minutes).

8.3 The Responsive Programme Review Web Interface

The system serves a dedicated web interface at `GET /programme-review?token=<TOKEN>`:

- **Token-Based Security**: Accessed via HMAC-SHA256 signed review tokens (`programmeReviewToken_()`). Does not require password login.
- **Zero-Config Client Bridge**: `src/server.js` injects a browser fetch bridge for `google.script.run`, allowing `ProgrammeReview.html` to function identically in standalone web browsers and Google Apps Script iframe environments.
- **Actions Supported**: Keep Open, Hold for Session, Request Speaker Move, Confirm Session (with Title, Presenter, Duration), and Release Slot.

9. Operational Timeline: 3-Week Meeting Lifecycle Walkthrough

This timeline illustrates how the automation engine autonomously coordinates a meeting across 21 days:

T-21 Days — **Long-Range Programme Review & Slot Booking** — Phase: Planning

- System evaluates speaking history and invites candidate members for speech slots.
- VPE reviews the 60-day horizon via `/programme-review`, placing holds or confirming special sessions.

T-14 Days — **Role Vacancy Announcement & Self-Selection** — Phase: Role Allocation

- `processVacantRoleReminders_()` identifies open roles.
- Broadcasts `atk_role_reminder_v2` template to active members.
- Members click `[View roles]` and claim roles interactively via WhatsApp.

T-7 Days — **Speaker Details & TMOD Confirmation** — Phase: Content Assembly

- System prompts confirmed speakers for Pathway, Level, Project, Title, and Time.
- Prompts TMOD for meeting theme.
- TMOD replies with theme ➔ System cascades theme to all assigned role players & VPPR.

T-4 Days — **Grammarian Collection & Role Introductions** — Phase: Language & Bios

- Grammarian conversational flow initiates: captures Word of the Day, then Idiom of the Day.
- Grammarian confirms package ➔ System delivers grammar package to TMOD.
- Role players prompted for introductions (custom text or default bio fallback).

T-2 Days — **Evaluator Matching & TMOD Handoff** — Phase: Handover

- Speech details sent to designated Speech Evaluators.
- Complete meeting introduction package handed off to TMOD for stage transitions.
- System compiles draft agenda.

T-1 Day — **Final VPE Agenda Approval & Meeting Day** — Phase: Execution

- Final readiness score calculated (Status.js).
- VPE reviews and confirms final agenda.
- Automated reminders dispatched to role players with venue and arrival instructions.

10. Dev Master Regression Suite, Testing & Troubleshooting

10.1 The 102-Point Automated Regression Matrix

To guarantee **100% exact logic parity** between the original Google Apps Script implementation and the Node.js / Express engine, a comprehensive regression suite runs via `npm test` (or `GET /api/dev/regression`):

Test Suite Name	Checks	What It Validates
Grammarian Parser	18 Checks	Validates multiline parsing, lowercase/uppercase headers, missing field detection, partial input recovery, and approval button reply routing.
Planner API Lifecycle	22 Checks	Validates `meeting.create`, item reordering, drag-and-drop preservation, draft saving, confirmed saving, and validation rules.
Core End-to-End Engine	26 Checks	Validates WhatsApp webhook challenge verification, inbound message deduplication, role menu generation, and outbox delivery logging.
Configured Role Workflows	18 Checks	Validates full role workflows under default configuration flags (Timer, TMOD, Grammarian, Evaluators).
Forced-Enabled Role Workflows	18 Checks	Forces all workflow flags to `TRUE` and verifies that every role branch correctly generates outbox prompts and completes tasks.

10.2 Environment Configuration Reference (.env)

Key	Example Value	Purpose & Effect
`PORT`	`3000`	HTTP port for Express web server.
`MONGODB_URI`	`mongodb+srv://...`	MongoDB Atlas cloud connection string.
`SPREADSHEET_ID`	`1qZnZov1l9...`	Workspace scope identifier for sheet replication.
`VERIFY_TOKEN`	`ATK_VPE_VERIFY_...`	Meta WhatsApp webhook verification secret.
`WEB_API_KEY`	`DEV_VPE_KEY_2026`	Secret key for Flutter Web API Gateway requests.
`SYSTEM_MODE`	`DEV` / `PROD`	System mode. Controls regression guards and diagnostic endpoints.
`WA_SEND_MODE`	`TEST` / `LIVE`	When `TEST`, outbound messages only go to allowlisted test phones.
`WA_TEST_RECIPIENTS`	`919846000000,...`	Comma-separated allowlist of developer/tester phone numbers.
`META_ACCESS_TOKEN`	`EAAPeNx...`	Meta Cloud API System User Bearer token.
`WA_PHONE_NUMBER_ID`	`1356900327505497`	WhatsApp Cloud API sender phone ID.
`WA_WABA_ID`	`462639891767985`	WhatsApp Business Account ID for template management.

Operational Summary & Verification

The ATK VPE Automation platform provides a complete, modern, high-speed solution for Toastmasters operations. By uniting WhatsApp conversational ease with robust cloud persistence and responsive web interfaces, it eliminates administrative burden while delivering a superior experience to club members and leaders alike.